

Optimizing the fleet's trajectory, not its next decision.

A supervisory layer that forecasts the fleet forward, simulates many coupled futures under constraints, and returns one policy for the existing stack to execute.

Aurelius sits above the scheduler. It does not replace Kubernetes, Slurm, or a serving router, and it never touches payloads. Each control period it forks the current fleet state into many forecasted futures, applies a coupled policy to each, advances them under SLA, capacity, power, and policy constraints, discards the ones that violate, and hands the single best safe policy back to the stack to run.

+724% average SLA-safe goodput per dollar

MEAN OF +698% / +718% / +755% (PJM / ERCOT / CAISO) VS A PRODUCTION-CLASS SCHEDULER BASELINE · UNCAPPED REPLAY OF PUBLIC PRODUCTION TRACES (~1.5M REQUESTS) · PARETO-SAFE AT ~84% FEWER GPU-HOURS

A number that size demands an explanation, and the report gives two. The **regime**: at production-scale load a fixed-policy scheduler degrades while a trajectory optimizer adapts. The **mechanism**: holding the model, objective, and safety gate fixed and changing only the search, a coupled search matches exhaustive enumeration at **zero measured regret**. The result is not simulator optimism; it is provable search.

00 Abstract

Conventional infrastructure stacks, including the unified control planes that consolidate a fleet into one view, optimize the next decision: the next placement, the next scale action, the next route, each chosen locally against the state visible now. Aurelius changes the unit being optimized. It treats the fleet as a system with a trajectory, forecasts that trajectory forward, and optimizes the path rather than the step.

Concretely, it is not a forecaster and not a scheduler replacement. It is a supervisory layer that forks the current fleet state into many forecasted futures, applies coupled policies across the controls the stack already exposes, advances each future under hard constraints, kills the ones that violate, and returns one policy for the existing scheduler to execute.

Replaying public production traces uncapped, at full volume (about 1.5M requests across the PJM, ERCOT, and CAISO expensive-power windows), Aurelius averages **+724% higher SLA-safe goodput per dollar** than a production-class scheduler baseline (per market +698% / +718% / +755%), while holding SLA strictly better and using **~84% fewer GPU-hours**. That is a Pareto win, not goodput bought by spending more.

A result that large is only worth reading if it is explained and defended, so this report does both. §07 gives the regime that produces the magnitude, a fixed policy degrading under uncapped load while a trajectory optimizer adapts. §05 gives the mechanism that proves it is real: with the world model, simulator, objective, and safety gate all held byte-identical, changing only which coupled futures the planner evaluates turned an SLA-breaking regression into a Pareto-dominant win that **matches exhaustive search at zero measured regret**. The value was not in the model or the objective. It was in which coupled futures got evaluated.

Scope, stated once and meant throughout: these are deterministic simulated replays on public production traces. The headline ratio is load-dependent and is quoted only with its scope (§07). This is directional evidence that requires live-telemetry calibration before any production claim, not a deployment.

01 The shift is in the unit of optimization

Most schedulers, and the control planes above them, optimize a current-state or local objective: availability, fairness, utilization, latency, queue state, immediate capacity, immediate price. Each is reasonable in isolation. But the economic outcome of a decision is set by constraints that arrive **after** it is made: power and capacity headroom, congestion, memory and topology pressure, demand, and the price of power, which on wholesale markets can move several-fold within a day.

Consolidating telemetry into one control plane is necessary, and it is not the missing piece. A unified present answers the question what is the best next decision given everything I can see now. It does not answer what is the best sequence of decisions given what is about to happen. Those are different questions, and the second is where the compounding waste lives, because once a job lands its costs are largely locked in and an observe-then-decide loop cannot price a constraint it has not yet seen.

Aurelius closes that gap by moving from an **observe, decide** loop to an **observe, forecast, simulate, decide** loop. It is a different control architecture, not another scheduler, and it changes the question from the next decision to the trajectory.

02 What Aurelius does each control period

Every control period, Aurelius forecasts the workload and conditions, forks the current fleet state into many candidate futures, applies a coupled policy to each, advances each future one control period at a time against the forecasted constraints, rejects any that fail a hard gate, scores the survivors by risk-adjusted SLA-safe goodput per dollar, and returns the single best safe policy for the existing scheduler to execute. When forecast confidence is low, it falls back deterministically to the stack's default.

The word that carries the architecture is **coupled**. Aurelius does not tune one knob. Workload timing, placement, routing, capacity, batching, and the precision and clock policy are scored together as one bundle, because those decisions interact, and, as the finding shows, the interaction is exactly where the economic value lives.

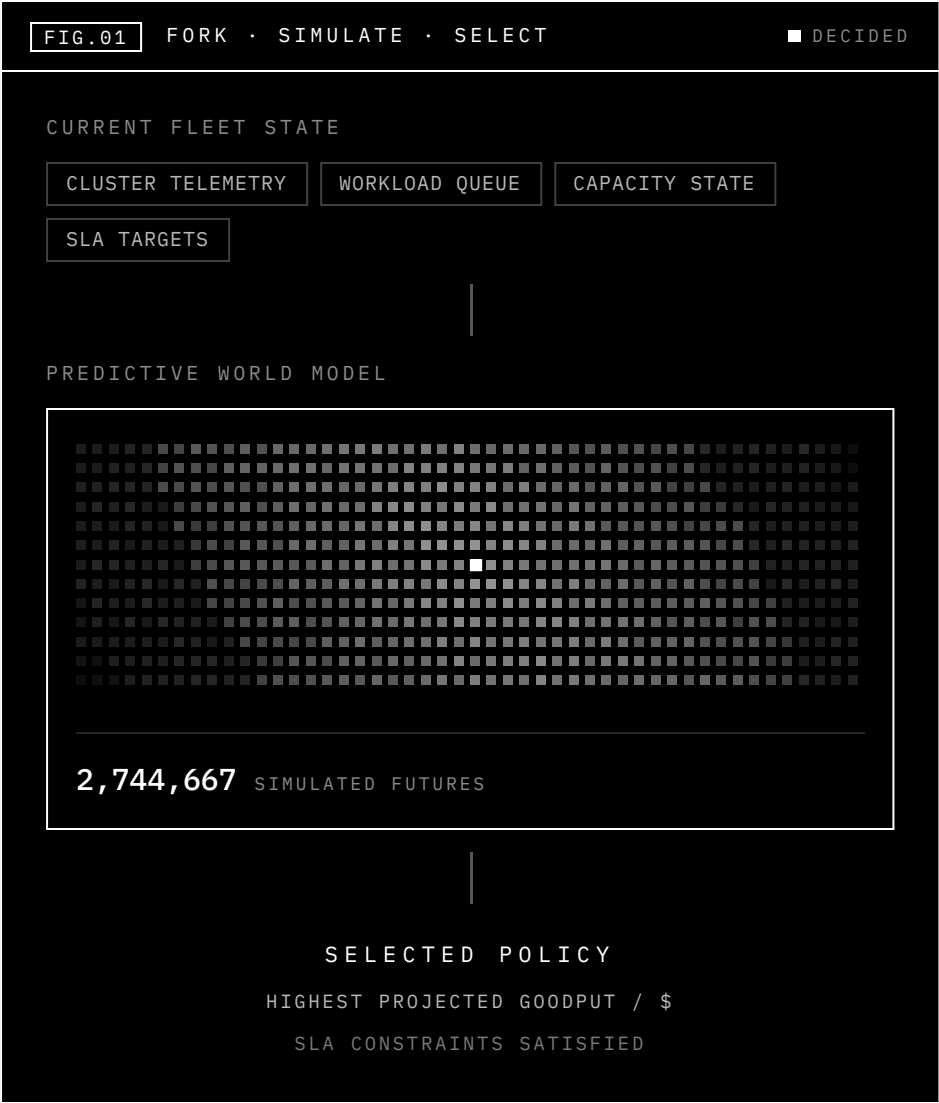


FIG.01 · ONE PLANNING CYCLE: MANY COUPLED FUTURES FORKED, ADVANCED UNDER CONSTRAINTS, ONE RETURNED

03 The predictive world model

Aurelius maintains a persistent world model of the cluster: a forward model of the fleet, not a snapshot of free GPUs. It carries replica warm and cold state, server and rack capacity, placement and locality, queue dynamics, in-flight migrations, and the operator-cost basis, and it advances that state one control period at a time so a candidate decision is scored against the state it will actually meet.

Candidate policies are evaluated on read-only simulated future states, so scoring never mutates the live cluster. Every input is tagged by provenance, measured, trace-derived, benchmark-calibrated, or modeled, so no result silently treats a modeled value as a measured one.

04 Which futures get evaluated is a first-class step

Because the controls interact, Aurelius evaluates coupled bundles rather than tuning one knob, and which bundles it evaluates is itself a first-class step, not an afterthought. A forecast-guided generator proposes a focused set of high-value coupled bundles implied by the operating regime, always including the known-strong candidates so a high-value option is never silently dropped. The set is then searched in a way that captures cross-surface coupling, widening to more surfaces only when a decision is close and stopping early when the choice is clear.

Where exact enumeration is tractable, the bounded search is scored against it, so any regret is **measured** rather than assumed. That discipline is what surfaced the finding in §05, and it is what lets a several-fold headline be defended rather than asserted.

ACTIVE DECISION LEVERS SCORED AS COUPLED BUNDLES; EACH CHANGES THE PROJECTED ECONOMIC OUTCOME

Capacity policy · Capacity multiplier · Ordering · Admission · Batching · Routing · Prewarm · Placement · Migration · Precision · Clock / DVFS

The finding is narrow, and that is why it is strong. With the world model, the simulator, the objective, and the safety gate all byte-identical, we changed only the candidate set and the search. Anything that moved is therefore attributable to the search, not to a model quietly made more optimistic. This is the check a skeptical reader should run first, because it is the one that separates a real result from a favorable simulator.

- **A single-lever search regressed.** Searching the GPU clock alone scored **-4.47%** against the baseline and made SLA worse, failing the safety gate. The optimizer cannot select a bundle it never evaluates.
- **Restoring the right coupled set fixed it.** With nothing else touched, proposing the high-value coupled bundles moved the same window to **+100%** and held SLA. The moment the winning bundle was in the set, the unchanged simulator picked it.
- **Coupled search added the rest.** Letting the search reach the coupled combinations a fixed grid does not contain lifted it to **+161%** goodput per dollar, at lower cost, 25% fewer GPU-hours, and 87% fewer SLA violations. A Pareto win, not goodput bought by spending more.
- **It left nothing on the table.** Against the exhaustive ground truth, where we could enumerate it, the bounded search matched the safe optimum **exactly, at zero measured regret**, in about 40 evaluations per decision. The direction holds across all three markets (+148% to +191%, zero regret in each).

Because everything downstream of the candidate set was held constant, the gain is attributable to the search alone, and the zero-regret check against enumerated ground truth is the proof that the optimizer is not leaving value on the table or inventing it. The residual gap to a forecast oracle on the same window is about 4.5%, and it is forecast quality, not search.

One exclusion, stated plainly. An aggressive low-precision (int4) mode scores higher still, but its quality cost is not yet modeled, so it is excluded from every headline and reported only as a labeled diagnostic ceiling. The headline-safe result uses the precision band whose quality we can defend.

06 The headline result: uncapped, production-scale replay

The headline replays the full selected windows with the per-period request cap removed, so each runs at its natural, production-scale volume: PJM ~577k, ERCOT ~443k, CAISO ~530k requests, about 1.5M in total. Both policies run the identical load. The bar is a **production-class scheduler baseline**, not a strawman: continuous (iteration-level) batching, SLA-aware ordering with deadline protection, KV and prefix-cache-aware routing, topology-aware placement, backlog autoscaling with a warm-pool headroom, and class-based admission. Each lever maps to an established production technique, so the composite is as close to a deployed scheduler as a public replay allows.

At that volume the naive SLA-aware policy does not complete: its no-batching, no-admission replay cost grows super-linearly and times out, so it is a reference only. The production-class scheduler and Aurelius both complete, which makes them the primary, production-comparable comparison: the policy built from real serving-stack components is the bar Aurelius has to beat.

TABLE 1 · UNCAPPED HIGH-LOAD REPLAY · AURELIUS VS PRODUCTION-CLASS SCHEDULER

MARKET	AURELIUS GP/\$	PRODUCTION GP/\$	Δ %	SLA (AUR.)	SLA (PROD.)	REQUESTS
PJM · expensive	1,041,842	130,539	+698.1%	0.0023	0.0382	576,912
ERCOT · expensive	1,064,602	130,178	+717.8%	0.0013	0.0447	442,716
CAISO · expensive	1,111,915	130,000	+755.3%	0.0018	0.0458	529,621
Average / total	–	–	+723.7%	–	–	1,549,249

gp/\$ and SLA-violation rates read directly from run output. Δ% is per market (Aurelius – production) ÷ production; the +724% headline is the equal-weighted average of the three (723.7%, shown unrounded). SLA violations fall from ~3.8–4.6% to ~0.1–0.2%.

TABLE 2 · GPU-HOURS AT THE SAME UNCAPPED LOAD

MARKET	AURELIUS GPU-H	PRODUCTION GPU-H	Δ GPU-HOURS
PJM · expensive	62.2	399.9	–84.4%
ERCOT · expensive	51.0	323.7	–84.3%
CAISO · expensive	60.0	391.5	–84.7%
Total	173.1	1,115.1	–84.5%

Same served requests both arms; the reduction is (production – Aurelius) ÷ production. Aurelius earns more SLA-safe goodput on far fewer GPU-hours because the coupled bundle packs more work per GPU-hour while holding SLA, which is why the win is Pareto and not paid for in spend.

07 How to read a several-fold number

A 7× gap over a competent scheduler is unusual, so the report is explicit about where it comes from and where its edges are. It is not the baseline being weak, and it is not the model being generous. It is the regime.

Why it is this large. Under uncapped, production-scale load a fixed-policy scheduler degrades super-linearly: it serves heavy load worse, its goodput per dollar falls toward a floor, and the naive SLA-aware policy stops completing at all. A trajectory optimizer does the opposite. It forecasts the squeeze, adapts the coupled bundle, and saturates. The gap between them is the cost of optimizing the next decision instead of the trajectory, and it is widest exactly when the fleet is most stressed. That asymmetry is the architecture showing itself, not an artifact.

Where its edges are. The ratio is load-dependent. It moves roughly five-fold with the per-period request cap, and at a frozen, comparable cap the same comparison is about **+140% to +160%**. So the exact multiple is a property of the load, and we quote it only with its scope. What is not load-dependent, and what actually carries the claim, is the direction and the zero-regret proof in §05: a trajectory optimizer and a step optimizer diverge under load, the divergence grows with stress, and the optimizer provably captures the safe optimum rather than a lucky point.

Read the several-fold number as two things at once: a stress signal, showing how far a fixed policy falls behind when the fleet is loaded, and a direction, made trustworthy by the zero-regret ablation. It is not a fleet constant, and this report never quotes it as one.

08 What is actually new here

The individual pieces are not new, and we do not claim they are. Forecasting, model-predictive control, digital twins of infrastructure, SLO-aware scheduling, and unified control planes all exist and are well understood. Aurelius's claim is narrower and, we think, more durable.

The unit of optimization is the fleet's **forecasted trajectory**, evaluated as coupled futures rather than independent next-decisions, and the binding constraint on the result is not the model or the objective but **which coupled futures get evaluated**. The finding in §05 is the evidence: on a byte-identical stack, changing only the candidate set and the search moved the outcome from an SLA-breaking regression to a Pareto-dominant win that provably matches exhaustive enumeration. That is a statement about where the value lives in fleet optimization, and it is testable on any fleet that keeps historical scheduler metadata.

09 Safety and the deployment path

Aurelius reads only the metadata a scheduler already exposes, job timing, resource requests, constraints, never payloads or model outputs. Candidate scoring runs read-only. Unsafe candidates are rejected at the constraint gate before any recommendation exists, and when confidence is low the controller falls back deterministically to the stack's default.

The path from telemetry to any production change is staged and reversible: offline replay of historical decisions, then read-only shadow mode alongside the live scheduler, then a controlled rollout on workloads explicitly in scope. Aurelius owns the **decision**; the customer's platform owns the **execution**. The blast radius of any Aurelius fault is therefore a suboptimal-but-valid placement, never an unsafe action, because the existing scheduler is always the fallback.

10 Limitations

- Every number here is deterministic simulated replay of public traces: directional evidence of achievable savings, not a guarantee for any specific fleet, and not a production deployment.
 - The +724% headline is load-dependent. Part of the gap is a fixed-policy scheduler degrading under heavy uncapped load while Aurelius adapts, so it is reported only with its uncapped, full-window scope and never as a fleet constant. At a frozen comparable cap the figure is ~+140–160%.
 - Magnitudes are simulator-inferred and depend on the serving model's precision and batching bands. The most robust findings are the **direction** and the **zero measured regret**, not any single percentage.
 - Aggressive low-precision is excluded from every headline until its quality cost is modeled.
 - Simulator fidelity must still be tested against real operator telemetry, which is the only way to isolate model error, and the largest gains appear during expensive-power windows.
 - Some controls are modeled but not yet active production levers, and several constraints are represented but not yet forecast inputs.
-

11 What we would validate with a partner

The next step is not a bigger simulation, it is calibration against reality. Given read-only historical scheduler metadata from a production fleet, we would replay its own decisions, simulate the coupled counterfactuals, and produce an audited estimate of the SLA-safe goodput per dollar left on the table, with no production changes and no payload access.

That is the test of whether the trajectory-level result generalizes beyond public traces, and it is the fastest, lowest-risk way to find out. If it holds on a real fleet's own history, it is a new axis of optimization for that fleet. If it does not, the replay says so, cheaply, before anything touches production.

A Appendix · The ablation, in full

The mechanism is clearest on the bounded window where exhaustive enumeration is tractable and the gain can be attributed exactly. Reward, cost model, and Pareto gate are byte-identical across every arm; the only thing that changes is which candidates the planner evaluates. The zero-regret column, measured against the enumerated safe optimum, is the proof that the bounded search forfeits nothing, and it is the same mechanism that drives the uncapped headline in §06.

TABLE A1 · CANDIDATE SET AND SEARCH VS THE EXHAUSTIVE GROUND TRUTH (PJM EXPENSIVE WINDOW)

ARM	Δ GOODPUT/\$	SLA VS BASELINE	EVALS	REGRET VS EXHAUSTIVE
Single-lever (clock only)	-4.47%	worse, fails gate	4	173.5% forfeited
Coupled candidate set (argmax)	+100.48%	better	18	30.3%
Coupled set + bounded search	+161.31%	better	~40	0%
Exhaustive (safe ground truth)	+161.31%	better	55	reference

The bounded search reaches the exact exhaustive-safe optimum (814,383 gp/\$ in both arms) at roughly 40 evaluations against 55 for full enumeration. An int4 arm scores higher (+226.88%) but carries unmodeled quality risk and is excluded from every headline.

TABLE A2 · DIRECTION ACROSS THREE INDEPENDENT ELECTRICITY MARKETS

MARKET	SINGLE-LEVER	COUPLED + SEARCH	SLA (COUPLED)	REGRET VS EXHAUSTIVE
PJM·expensive	-4.47%, SLA worse	+161.31%	better	0
ERCOT·expensive	-10.31%, SLA worse	+191.41%	better	0
CAISO·expensive	+0.50%, SLA worse	+147.85%	better	0

Single-lever never passes the Pareto gate in any market (SLA always worse). Coupled search is Pareto-dominant in all three and matches the safe exhaustive optimum exactly in each. Simulated; bounded to these windows.

References

The production-class scheduler baseline (§06) is assembled from established production serving and cluster-scheduling techniques. These are cited for the realism of the baseline mechanisms, not for any Aurelius result.

- [1] Yu et al. “Orca: A Distributed Serving System for Transformer-Based Generative Models.” OSDI 2022. Iteration-level (continuous) batching.
- [2] Kwon et al. “Efficient Memory Management for Large Language Model Serving with PagedAttention” (vLLM). SOSP 2023. Continuous batching and KV-cache management.
- [3] Gujarati et al. “Serving DNNs like Clockwork: Performance Predictability from the Bottom Up.” OSDI 2020. SLO-aware scheduling and admission.
- [4] Zheng et al. “SGLang: Efficient Execution of Structured Language Model Programs” (RadixAttention). 2024. Prefix / KV-cache-aware reuse.
- [5] SchedMD. “Slurm Workload Manager, Topology Guide.” Topology-aware placement in production cluster schedulers.
- [6] Rządca et al. “Autopilot: Workload Autoscaling at Google.” EuroSys 2020. Utilization-headroom autoscaling at production scale.